

Dispatch Service

Distributed Task Execution System

Modern C++23 Implementation
Phase 15 Complete

Active Development

This document provides a comprehensive overview of the Dispatch Service architecture, implementation, and learning objectives.

Latest Update: Phase 15 Complete

Contents

1	Project Purpose and Scope	3
2	System Architecture	4
2.1	Three-Tier Design	4
2.2	Message Types	4
2.3	Execution Flow	4
3	Protocol Layer	6
3.1	Wire Format	6
3.2	Transport Layer (transport.h)	6
3.3	Protocol Layer (protocol.h)	6
4	Client Implementation	7
4.1	State Machine	7
4.2	Client CLI (client_cli.cpp)	7
4.3	Execution Engine	7
5	Worker Implementation	9
5.1	Worker Runtime	9
5.1.1	WorkerPool	9
5.1.2	Worker Interface	9
5.2	Task Registry (Task_Registry.h)	9
5.3	Workload Types	10
5.4	Worker Profiles	10
6	Reliability Mechanisms	11
6.1	Timeouts	11
6.2	Retries	11
6.3	At-Least-Once Delivery	11
6.4	Idempotent Execution	11

7	Development Phases	12
8	Code Organisation	13
9	Build System	14
9.1	CMake Configuration	14
9.2	Building	14
10	Running the System	15
10.1	Start Worker	15
10.2	Run Client	15
10.3	Run Scenario Tests	15
11	Scenario Testing	16
11.1	HappyPath	16
11.2	TimeoutRetry	16
11.3	CachedResult	16
11.4	ConcurrentClients	16
11.5	SpedUpWorkers	16
12	Current Capabilities	17
12.1	Working	17
12.2	Not Yet Implemented	17
13	Known Limitations	18
13.1	In-Memory State	18
13.2	Retry Logic Coupling	18
13.3	Socket Resource Leaks	18
13.4	Worker Scheduling	18
14	Future Roadmap	19
14.1	Short Term	19
14.2	Medium Term	19
14.3	Long Term	19
15	Learning Objectives Achieved	20
15.1	Systems Thinking	20
15.2	Distributed Systems	20
15.3	Modern C++	20
15.4	Concurrency	20

Conclusion

22

Chapter 1

Project Purpose and Scope

This project studies distributed systems engineering through a task execution platform. The goal is to understand networking, concurrency, reliability, retries, error handling, and distributed system behaviour.

The system submits tasks from clients to a dispatcher, which routes them to workers for execution. Tasks have built-in retry logic and timeout handling. Workers track task execution state to prevent duplicate execution even when the network delivers the same task multiple times.

This is not a clone of Temporal. Rather, it focuses on prerequisite layers that any orchestration system requires: protocol design, message framing, concurrency safety, and idempotent execution.

Chapter 2

System Architecture

2.1 Three-Tier Design

The system follows a clean three-tier architecture:

Client —_i Dispatcher —_i Worker

Client: Initiates task submission. Waits for acknowledgement and results. Retries on timeout.

Dispatcher: Receives task submissions. Sends acknowledgement. Routes to worker. Manages timeouts and retries.

Worker: Accepts TCP connections and enqueues work items. A fixed-size WorkerPool owns worker threads which dequeue connections and execute the worker protocol pipeline.

2.2 Message Types

Three core message types flow through the system:

TASK_SUBMIT (client to dispatcher/worker): Contains task ID, idempotency key, payload, and workload type.

TASK_ACK (dispatcher/worker to client): Acknowledges receipt of TASK_SUBMIT immediately.

TASK_RESULT (worker to dispatcher to client): Contains task status, payload (result data), and timing information.

2.3 Execution Flow

The typical request lifecycle:

1. Client creates socket, connects to dispatcher
2. Client submits task (TASK_SUBMIT)
3. Dispatcher sends ACK (TASK_ACK)
4. Dispatcher routes task to worker

5. Worker marks task as “Processing”
6. Worker executes task
7. Worker marks task as “Completed” with result
8. Dispatcher relays TASK_RESULT to client
9. Client receives result and closes connection

If timeout occurs before TASK_RESULT, client retries. Dispatcher or worker detects already processing or cached state.

Chapter 3

Protocol Layer

3.1 Wire Format

All messages use length-prefixed framing to handle TCP's lack of message boundaries:

Frame: [4-byte length][1-byte type][payload]
--

The 4-byte length indicates bytes that follow (including type byte).

3.2 Transport Layer (transport.h)

The transport layer sits below protocol and knows nothing about tasks.

SendFrame(socket, type, payload): Wraps payload in frame. Loops over partial writes. Returns false on error.

RecvFrame(socket): Reads 5-byte header. Allocates payload buffer. Loops over partial reads. Returns std::optional.

RecvExact(socket, buffer, n): Loops until exactly n bytes are read.

3.3 Protocol Layer (protocol.h)

Sits above transport and interprets frames as task messages.

MessageType enum: kTaskSubmit, kTaskAck, kTaskResult, kCancel.

TaskStatus enum: kPending, kRunning, kSucceeded, kFailed, kInProgress.

Chapter 4

Client Implementation

4.1 State Machine

Client uses state machine with explicit states for each protocol step.

States: InitSocket, ConfigureAddress, ConfigureTimeout, Connect, SubmitTask, WaitAck, WaitResult, RetryDecision, CloseSocket, BackOff, Success, Failure.

Each state is C++ type. Single **step()** function overloaded for each type.

Monadic Error Handling: Uses `std::expected`. Each step returns new state or error. Allows chaining with `.and_then()` and `.transform()`.

4.2 Client CLI (`client_cli.cpp`)

Simple synchronous client dispatching single task:

1. Create socket
2. Set server address to 127.0.0.1:50051
3. Set receive timeout to 1500ms
4. Connect
5. Submit task (task-0001)
6. Wait for ACK
7. Wait for result
8. Close socket

On timeout, retries once. On success or failure, terminates.

4.3 Execution Engine

Responsible for building client contexts, launching scenario clients, aggregating results, returning scenario outcomes.

Supported scenarios: HappyPath, TimeoutRetry, CachedResult, ConcurrentClients, SpedUp-Workers.

Chapter 5

Worker Implementation

5.1 Worker Runtime

Worker runtime owns listening socket and accepts TCP connections. Each connection converts to work item submitted to WorkerPool.

5.1.1 WorkerPool

Owens: fixed-size worker thread set, shared work queue, queue mutex, condition variable, graceful shutdown state.

Workers remain idle until work available.

5.1.2 Worker Interface

Connection processing isolated in worker_interface.

Pipeline: ReceiveMessage, SubmitACK, Process, Execute, SendResult.

HandleConnection coordinates complete lifecycle.

5.2 Task Registry (Task_Registry.h)

Mutex-protected data structure tracking execution state per task ID.

Task States:

- Absent: Not in registry
- Processing: Task currently executing
- Completed: Task executed, result cached

Operations:

`try_claim(taskId)`: Acquires mutex, checks taskId.

- If absent: Add to registry with Processing status. Return Execute.
- If Processing: Return Reject (duplicate in flight).

- If Completed: Return Cached with result.

`mark_complete(taskId, result)`: Changes from Processing to Completed, stores result.

5.3 Workload Types

Six workload types for testing:

- SlowSuccess: Sleeps 2 seconds then succeeds
- FastSuccess: Succeeds immediately
- ImmediateFailure: Fails immediately
- DelayedFailure: Sleeps 2 seconds then fails
- RandomChance: Succeeds or fails randomly
- RandomDelay: Random sleep duration

5.4 Worker Profiles

Each worker owns Profile containing SpeedClass and duration_factor.

Speed Classes:

- Fast = 0.5
- Normal = 1.0
- Slow = 2.0

Execution duration adjusted: $\text{effective_duration} = \text{base_duration} \times \text{duration_factor}$.

Chapter 6

Reliability Mechanisms

6.1 Timeouts

Client sets `SO_RCVTIMEO` on socket. Socket option causes `recv()` return after timeout if no data.

Default: 1500ms

Trigger: No `TASK_RESULT` within 1500ms, `recv()` returns 0 (EOF), error `WSAETIMEDOUT`.

Response: Interpreted as timeout, may retry.

6.2 Retries

Client maintains retry counter. On timeout, increments counter, retries if below threshold.

Current Design: Retries once (two attempts). After second timeout, gives up.

6.3 At-Least-Once Delivery

Client retries on timeout, same task submitted multiple times. Dispatcher/worker relies on application layer for deduplication.

6.4 Idempotent Execution

Worker implements idempotency via task registry. Second submission of same task returns cached result without re-executing.

Guarantee: Exactly one execution on worker, regardless of `TASK.SUBMIT` count.

Limitation: Holds only within single worker. True cluster deduplication requires coordination.

Chapter 7

Development Phases

System progressed through 15 phases:

Phase 1–3: Protocol layer, standalone/concurrent worker.

Phase 4–6: Client state machine, timeouts, retries.

Phase 7–9: Delivery, idempotent execution validation.

Phase 10: Duplicate suppression validation.

Phase 11–12: Execution engine, concurrent clients.

Phase 13–14: Worker interface refactor, WorkerPool.

Phase 15: Worker profiles, heterogeneous speeds.

Chapter 8

Code Organisation

```
src/  
  main.cpp  
  client_cli.cpp  
  worker_cli.cpp  
  utility/  
    Wire/  
      transport.h  
      protocol.h  
      Task_Registry.h  
  worker_pool/  
  workload/  
  client/  
  scenario/  
  execution/  
  random/  
  scope_logger/
```


Chapter 9

Build System

9.1 CMake Configuration

Requires C++23 compiler (GCC 14+, Clang 17+, MSVC 2022+).

```
cmake_minimum_required(VERSION 3.20)
project(dispatch_service_smoketest CXX)

set(CMAKE_CXX_STANDARD 23)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

9.2 Building

```
mkdir build && cd build
cmake ..
cmake --build .
```

Outputs: build/worker, build/client, build/smoke_test, build/logger.

Chapter 10

Running the System

10.1 Start Worker

```
./build/worker
```

Listens on 127.0.0.1:50051.

10.2 Run Client

```
./build/client
```

Connects, submits task-0001, waits for result.

10.3 Run Scenario Tests

```
./build/smoke_test
```

Select scenarios by uncommenting in main.cpp.

Chapter 11

Scenario Testing

11.1 HappyPath

Single task with FastSuccess. Verifies immediate completion.

11.2 TimeoutRetry

Task with SlowSuccess (2000ms), timeout 1500ms. Verifies timeout and retry.

11.3 CachedResult

Same task ID twice. Verifies second submission returns cached result.

11.4 ConcurrentClients

Multiple clients concurrent submission. Verifies no deadlock.

11.5 SpedUpWorkers

Heterogeneous pool: Fast/Normal/Slow workers at different speeds.

Chapter 12

Current Capabilities

12.1 Working

- TCP networking, platform abstraction
- Protocol framing, serialisation
- WorkerPool with graceful shutdown
- Concurrent clients via `std::future`
- Execution Engine orchestration
- Worker profiles, heterogeneous speeds
- Timeouts, retry logic
- At-least-once delivery, idempotent execution
- `std::expected` error handling
- Monadic chaining, scenario testing

12.2 Not Yet Implemented

- Dispatcher component (integrated with worker)
- Multi-worker routing
- Socket RAI wrappers
- Shutdown signal handling
- Persistent task state
- Automatic backoff
- Load balancing, health checks
- Distributed idempotency
- Task cancellation

Chapter 13

Known Limitations

13.1 In-Memory State

Registry never persisted. Restart loses history. Breaks idempotency.

13.2 Retry Logic Coupling

Embedded in client state machine. Difficult to test isolation.

13.3 Socket Resource Leaks

Not RAII wrapped. Defer pattern used. Exception throw risks closure.

13.4 Worker Scheduling

Shared queue only. No work stealing, prioritisation, capability routing.

Chapter 14

Future Roadmap

14.1 Short Term

Advanced scheduling: heterogeneous configs, queue-aware routing. Workload orchestration for complex patterns.

14.2 Medium Term

Graceful shutdown: signal handling. Socket RAII. Exponential backoff.

14.3 Long Term

Persistence: KV store integration. Distributed idempotency: Redis registry. Exactly-once semantics. Health checks. Task cancellation. Performance instrumentation.

Chapter 15

Learning Objectives Achieved

15.1 Systems Thinking

Layers: Transport knows nothing about tasks. Protocol knows nothing about workers. Workers know nothing about routing. Separation enables independent layer testing.

15.2 Distributed Systems

- At-least-once delivery, idempotent execution
- State machines for interactions
- Timeouts, retry logic
- Duplicate suppression
- Task ownership, concurrency safety

15.3 Modern C++

- `std::expected` error handling
- Monadic chaining
- `std::jthread` concurrency
- RAI, scope guards
- C++23 features
- Platform abstraction
- Move semantics

15.4 Concurrency

- Producer-consumer queues

- Condition variables
- Worker pools
- Coordinated shutdown
- Work distribution

Conclusion

Dispatch Service studies distributed task execution hands-on. Focuses on foundational layers: reliable messaging, graceful failure handling, idempotent execution.

Phased development shows complex systems built from simple, tested layers. Each phase adds concern: networking, concurrency, timeouts, retries, idempotency. Combination yields reliable systems under realistic failure.

Next milestone—multi-worker routing—introduces real orchestration questions: work distribution, failure detection, recovery.

Dispatch Service

Phase 15 Complete — Active Development